KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Week 6 Laboratory Sheet

Modeling Languages I: MiniZinc and Solver Independence

Assoc. Prof. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Estimated time: 2 hours
Requirements: Terminal/shell access (macOS/Linux: Terminal, Windows: PowerShell), the course's shared Python 3.12 virtual environment from Week 1 (already carrying the `minizinc` package), a text editor, and network access for one-time installs of the MiniZinc CLI and the HiGHS solver library

Learning phases: 1 • Pre-instruction | 2 • Content | 3 • Participation | 4 • Assessment | 5 • Follow-through

1 • Pre-instruction — Why It Matters

Every model you have built so far this course lived inside a single Python program tied to one solver, OR-Tools CP-SAT. Today you write a model in a language whose only job is describing *what* a solution must satisfy, then hand that exact model, unedited, to three different solvers — a CP solver and two MIP backends — and watch all three agree on the same optimal answer despite finding it by genuinely different methods. Along the way you also catch a solver that is *listed* as available but does not actually work, and you close by building one such model completely on your own.

1 • Pre-instruction — Prior Knowledge

This sheet exercises all four units of this week's chapter: the parameter/variable/constraint/solve structure of a `.mzn` file (Unit 1), how a model is flattened into FlatZinc before any solver reads it (Unit 2), running one unchanged model on Gecode and on a MIP backend (Unit 3), and driving MiniZinc from Python with the `minizinc` package (Unit 4). It also reuses the exact N-queens formulation from Week 1. Keep the chapter open beside this sheet — each section below points to the unit it exercises.

Use this sheet as follows: in each step, complete the task described under `What will you do?`, then compare your own result against the `Expected result` box to confirm you are on track before moving on. Steps follow the chapter's four units in order, and a `Checkpoint` every few steps lets you confirm everything so far is working before you continue.

1 Setup: Installing MiniZinc and Confirming the Solver Inventory

Step 1: Install the MiniZinc CLI and verify its version

What will you do? You will install the MiniZinc command-line tool itself — new this week, separate from the `minizinc` Python package already pinned since Week 1 — and print its reported version.

Why? Every `.mzn`/`.dzn` pair you build or run today is compiled and solved by this binary,

Step 1 (continued)

not by the Python package alone; a version mismatch against the one this book's captured output comes from (MiniZinc 2.10.1) is the single most common source of a lab step that almost, but not quite, matches its `Expected result` box.

Terminal

```
$ brew install minizinc
$ minizinc --version
```

Expected result: The second command prints a line naming the tool and its version.

Expected Output

```
MiniZinc to FlatZinc converter, version 2.10.1
Copyright (C) 2014-2026 Monash University, NICTA, Data61
```

macOS

`brew install minizinc` pulls in MiniZinc together with its bundled Gecode 6.4 solver; this is the exact toolchain this book's captured output comes from, and the one every `Expected result` box in this sheet assumes.

Windows

Download the MiniZinc bundle installer from the official MiniZinc website and run it; the bundle already includes Gecode, so no separate solver install is needed for this sheet's Gecode steps. Your own reported version number and any solve-time figures may differ slightly from the macOS-captured output shown in this sheet.

Linux

Most distributions do not package a MiniZinc release recent enough for this book; download the official Linux bundle (a self-contained archive with its own install script) from the MiniZinc website rather than relying on your distribution's package manager.

Step 2: Install and configure the HiGHS backend

What will you do? `minizinc --solvers` already *lists* HiGHS as a registered MIP backend, but it cannot actually solve anything until you install the HiGHS library itself and tell MiniZinc where to find it. You will install HiGHS via Homebrew, then add one entry to MiniZinc's own preferences file.

Why? This is this week's first concrete example of a distinction the whole chapter leans on: being *listed* by `minizinc --solvers` and being *able to solve* are two different claims. HiGHS sits on the "listed but not yet usable" side until this one extra step is done — and skipping it produces a plugin-load error later in this sheet that looks nothing like a mistake in your own model.

Step 2 (continued)

Terminal

```
$ brew install highs
$ mkdir -p ~/.minizinc
$ cat > ~/.minizinc/Preferences.json << 'JSON'
{
  "solverDefaults": [
    ["org.minizinc.mip.highs", "--highs-dll", "/opt/homebrew/lib/libhighs.dylib"]
  ]
}
JSON
$ echo 'var 1..2: x; solve satisfy;' > probe.mzn
$ minizinc --solver HiGHS probe.mzn
```

Expected result: The final command prints a trivial solution, confirming HiGHS now solves rather than merely being registered.

Expected Output

```
x = 1;
----------
```

Caution

If `~/.minizinc/Preferences.json` already exists with other content (from an earlier week or another tool), merge the `solverDefaults` key into the existing file instead of overwriting it — the heredoc above assumes the file does not exist yet. On Windows or Linux, the HiGHS shared library's file name and path differ (`highs.dll` or `libhighs.so` in a different install location); this sheet's exact path is macOS/Homebrew-specific.

Step 3: Inventory the registered solvers and probe which ones actually solve

What will you do? You will run `minizinc --solvers` and read its output, then run the same trivial one-variable model from the previous step against each of the three MIP backend names it lists — `COIN-BC`, `HiGHS` and `Gurobi` — to confirm which ones actually solve here.

Why? This is the chapter's central environment fact, verified on your own machine rather than taken on faith: `minizinc --solvers` reports every solver *configuration* it knows about, which is not the same as every solver that can actually finish a solve. Unit 3 builds its entire solver-independence argument on Gecode, COIN-BC and HiGHS — never on Gurobi — precisely because of what this step shows you.

Terminal

```
$ minizinc --solvers
```

Expected result: Gecode and its `Gecode Gist` variant, plus four MIP backend *registrations* — COIN-BC, HiGHS, Gurobi, and a small number of others your local Homebrew install also registers a configuration stub for. Chuffed and any OR-Tools FlatZinc target must be absent from the list entirely — this course's environment never installs either.

Step 3 (continued)

Expected Output

```
MiniZinc driver.
Available solver configurations:
  COIN-BC 2.10.13/1.17.11 (org.minizinc.mip.coin-bc, mip, float, api,
      osicbc, coinbc, cbc)
  CPLEX <unknown version> (org.minizinc.mip.cplex, mip, float, api)
  Gecode 6.4.0 (org.gecode.gecode, default solver, cp, int, float, set,
      restart)
  Gecode Gist 6.4.0 (org.gecode.gist, cp, int, float, set, restart)
  Gurobi 12.0.3 (org.minizinc.mip.gurobi, mip, float, api)
  HiGHS 1.15.1 (org.minizinc.mip.highs, mip, float, api, highs)
  SCIP <unknown version> (org.minizinc.mip.scip, mip, float, api)
  Xpress <unknown version> (org.minizinc.mip.xpress, mip, float, api,
      float_times)
```

Terminal

```
$ minizinc --solver COIN-BC probe.mzn
$ minizinc --solver Gurobi probe.mzn
```

Expected result: Both COIN-BC here and HiGHS in the previous step print a trivial solution; the Gurobi attempt fails with a licence error — this course's shared environment never gets a working MiniZinc Gurobi backend.

Expected Output

```
x = 1;
----------
```

Expected Output

```
Error: error:   MIPGurobiWrapper runtime error:  No Gurobi license found
Message from caller: Could not open GUROBI environment.
```

Common Pitfall

`CPLEX`, `SCIP` and `Xpress` also appear in the solver list with `<unknown version>` — these are unconfigured placeholder registrations, not solvers this course installs or uses. Ignore them; the only four solver names that matter anywhere in this chapter are Gecode, COIN-BC, HiGHS and Gurobi, and only the first three of those four actually solve in this environment.

Checkpoint — verify before moving on

`minizinc --version` reports 2.10.1, HiGHS solves the trivial probe model after your `Preferences.json` edit, and your own three-backend probe reproduces this sheet's exact pattern: COIN-BC and HiGHS succeed, Gurobi fails with a licence error. If all three hold, your toolchain matches the one every remaining step in this sheet assumes.

2 Unit 1 — Declarative Modeling in MiniZinc

(See Course Notes Week 6, Unit 1 — Declarative Modeling in MiniZinc.)

Step 4: Formulate a fresh knapsack instance's declarations on paper

What will you do? A 0/1 knapsack has capacity 10 and five items, each with its own weight and value (call the parameter arrays `weight` and `value`, indexed `1..n` with `n = 5`, exactly as this week's `knapsack.mzn` does). On paper, write the MiniZinc decision-variable declaration for this instance and the constraint stating that the selected items' total weight may not exceed capacity — do not write the full model, and do not plug in specific numbers for the weights or values.

Why? This is the same two-line exercise Unit 1's own practice task walks through on its own knapsack instance; repeating it here, on a `n = 5` instance instead of Unit 1's `n = 4` one, is what turns "following Unit 1's declaration" into "writing one yourself whenever `n` and `capacity` change."

Expected result: `array[1..n] of var bool: x;` for the decision variables — one Boolean per item, true if selected — and `constraint sum(i in 1..n)(weight[i] * x[i]) <= capacity;` for the capacity constraint. Neither line changes with `n` or `capacity`'s concrete values, because both are parameters referenced by name, not hard-coded numbers — this is exactly what lets one `.mzn` file serve any `.dzn` instance.

Step 5: Read knapsack.mzn's four building blocks and solve it on Gecode

What will you do? Open `minizinc/knapsack.mzn` and locate its four parts — parameters, the decision-variable array, the capacity constraint, and the `solve` item with its `output` — shown below. Then run it on Gecode against the `n = 4` instance in `knapsack.dzn`.

Why? This is Unit 1's own worked knapsack model, not the fresh instance you just formulated on paper — running the real file, side by side with the four blocks you just read, is what confirms your paper declarations from the previous step describe exactly what this working file contains.

Parameters — fixed, known data supplied later by a `.dzn` file:

```
int: n;
int: capacity;
array[1..n] of int: weight;
array[1..n] of int: value;
```

The decision variables — one Boolean per item:

```
array[1..n] of var bool: x;
```

The capacity constraint:

```
constraint sum(i in 1..n)(weight[i] * x[i]) <= capacity;
```

The `solve` item, with an `output` item formatting the solved instance's own printout:

```
solve maximize sum(i in 1..n)(value[i] * x[i]);

output ["selected items: \(x)\n",
      "total value: \(sum(i in 1..n)(value[i] * x[i]))\n"];
```

```
Terminal
$ cd kod_ornekleri/hafta06
$ minizinc --solver Gecode minizinc/knapsack.mzn minizinc/knapsack.dzn
```

Step 5 (continued)

Expected result: Items 2 and 3 selected (weights 3 and 4, total 7, exactly at capacity), total value 9.

Expected Output

```
selected items: [false, true, true, false]
total value: 9
----------
==========
```

Tip

If your terminal is still at the course root, `cd kod_ornekleri` first, then `cd hafta06` — every remaining command in this sheet is run from inside `kod_ornekleri/hafta06`. Your own terminal also prints a one-line warning on stderr before the result (`included file "knapsack.mzn" overrides a global constraint file from the standard library`) — MiniZinc's own standard library ships a file with this exact name, so the collision is expected and harmless, not a bug in this week's model.

Checkpoint — verify before moving on

You have written a knapsack declaration and constraint entirely from a plain description, and watched the chapter's own `knapsack.mzn` solve a different instance on Gecode with the printed value your paper declarations would also produce for that instance. You are ready to see what happens to a model like this one before any solver reads it.

3 Unit 2 — FlatZinc and Model Transformation

(*See Course Notes Week 6, Unit 2 — FlatZinc and Model Transformation.*)

Step 6: Compile N-queens to FlatZinc and find the flattened constraint

What will you do? You will compile `minizinc/nqueens.mzn` (the exact Week 1–4 N-queens formulation, rewritten in MiniZinc) against `nqueens.dzn`'s `n = 6` without solving it, inspect the resulting `.fzn` file, and find the line corresponding to your `constraint all_different(q);` on the columns.

Why? This is Unit 2's own worked example — compiling a model you already trust (it is Week 1's N-queens CSP, unchanged in substance) is what lets you focus entirely on *what the compiler did to it* instead of also having to check whether the model itself is correct.

The declarations — one row-indexed variable per queen, domain `1..n`:

```
int: n;
array[1..n] of var 1..n: q;
```

The three constraints — columns, then both diagonal transformations:

```
constraint all_different(q);
constraint all_different([q[i] + i | i in 1..n]);
constraint all_different([q[i] - i | i in 1..n]);
```

Step 6 (continued)

Terminal

```
$ minizinc --compile -o nqueens.fzn --solver Gecode \
    minizinc/nqueens.mzn minizinc/nqueens.dzn
$ cat nqueens.fzn
```

Expected result: A FlatZinc file with no trace of the literal identifier `all_different` anywhere in it — the columns constraint appears instead as `fzn_all_different_int(q)`, and `n` has disappeared, replaced everywhere by the concrete number 6 from `nqueens.dzn`.

Expected Output

```
% Generated by MiniZinc 2.10.1
% Solver library: /opt/homebrew/Cellar/gecode/6.4.0/share/minizinc/gecode

predicate fzn_all_different_int(array [int] of var int: xs);
...
array [1..6] of var int: q :: output_array([1..6]) = [...];
array [1..6] of var int: X_INTRODUCED_13_ :: var_is_introduced = [...];
array [1..6] of var int: X_INTRODUCED_20_ :: var_is_introduced = [...];
constraint fzn_all_different_int(q);
constraint fzn_all_different_int(X_INTRODUCED_13_);
constraint fzn_all_different_int(X_INTRODUCED_20_);
...
solve  satisfy;
```

Common Pitfall

The `X_INTRODUCED_...` arrays are not a mistake in your `.mzn` file — they are the two diagonal transformations (`q[i] + i` and `q[i] - i`) that MiniZinc's flattener had to materialise as their own auxiliary variables before it could hand them to `fzn_all_different_int`, since that predicate expects a plain array of variables, not an arithmetic expression. You never named these variables; the compiler did, the same way an auxiliary variable you add by hand in Python would need its own name.

Checkpoint — verify before moving on

Your compiled `nqueens.fzn` contains `fzn_all_different_int(q)`, not the literal string `all_different`, and you can point to the auxiliary `X_INTRODUCED_...` arrays the two diagonal constraints needed. You are ready to see what changes, and what does not, when this same kind of file is handed to a different solver.

4 Unit 3 — Running One Model on Gecode and a MIP Backend, Unchanged

(See Course Notes Week 6, Unit 3 — Running One Model on Gecode and a MIP Backend, Unchanged.)

Step 7: Read assignment.mzn and solve it on Gecode with statistics

What will you do? Open `minizinc/assignment.mzn` — the assignment problem, this unit's worked example — and locate its parameters, decision variables, feasibility constraint

Step 7 (continued)

and `solve` item. Then run it on Gecode against the 4×4 cost matrix in `assignment.dzn`, with the `--statistics` flag.

Why? Unit 3 deliberately chooses the assignment problem over N-queens for this unit's running example because it has an objective: "the optimal *value* matches across solvers" is a concrete, checkable number here, not just "a solution exists."

The cost matrix parameter and the one decision variable per worker:

```
int: n;
array[1..n, 1..n] of int: cost;
```

```
array[1..n] of var 1..n: assign;
```

The feasibility constraint — no two workers share a task:

```
constraint all_different(assign);
```

Terminal

```
$ minizinc --solver Gecode --statistics \
    minizinc/assignment.mzn minizinc/assignment.dzn
```

Expected result: An optimal total cost of 13, followed by Gecode's own solving statistics — `nodes`, `failures` and a `solveTime` figure in the vocabulary of backtracking search.

Expected Output

```
assignment: [2, 1, 3, 4]
total cost: 13
----------
==========
%%%mzn-stat: initTime=0.000215584
%%%mzn-stat: solveTime=5e-05
%%%mzn-stat: solutions=5
%%%mzn-stat: variables=9
%%%mzn-stat: propagators=6
%%%mzn-stat: propagations=86
%%%mzn-stat: nodes=13
%%%mzn-stat: failures=2
%%%mzn-stat: restarts=0
%%%mzn-stat: peakDepth=3
%%%mzn-stat-end
```

Step 8: Run the identical file, unchanged, on COIN-BC and HiGHS

What will you do? Without editing `assignment.mzn` or `assignment.dzn` at all, run the exact same two files through `--solver COIN-BC` and then `--solver HiGHS`, both with `--statistics`, and compare each solver's reported total cost and statistics vocabulary against Gecode's run above.

Why? This is Unit 3's central claim, checked with your own hands: the same `.mzn`+`.dzn` pair, with *no edits*, handed to two solvers built on completely different algorithms — LP relaxation plus branch-and-bound instead of propagation plus backtracking search — and both must report the same optimal cost.

Step 8 (continued)

Terminal

```
$ minizinc --solver COIN-BC --statistics \
    minizinc/assignment.mzn minizinc/assignment.dzn
$ minizinc --solver HiGHS --statistics \
    minizinc/assignment.mzn minizinc/assignment.dzn
```

Expected result: Total cost 13 from both solvers — identical to Gecode's — but a different statistics vocabulary (`objectiveBound`, `nodes` in the branch-and-bound sense, no `failures`/`propagations` at all) and a different assigned pairing may or may not appear, since more than one assignment can tie for cost 13.

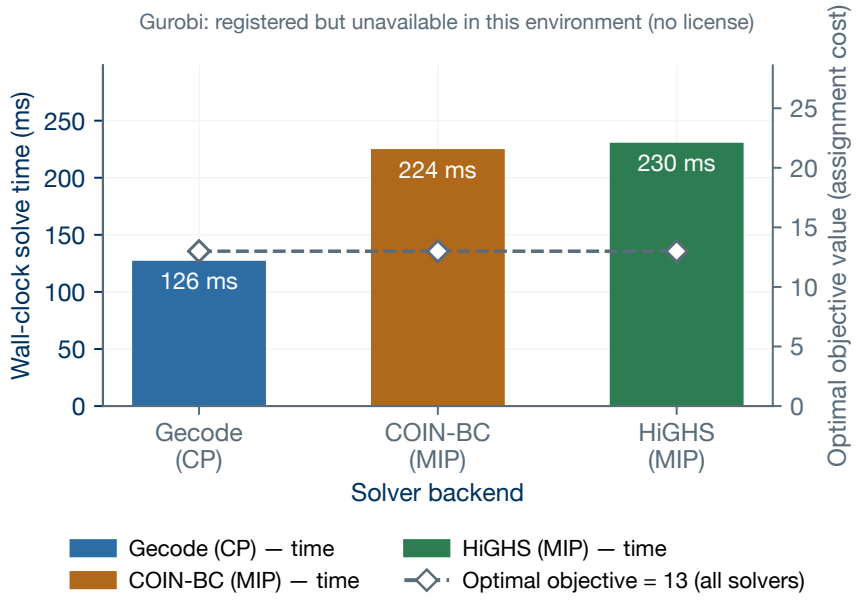
Expected Output

```
assignment: [2, 1, 3, 4]
total cost: 13
----------
==========
%%%mzn-stat: objective=13
%%%mzn-stat: objectiveBound=13
%%%mzn-stat: nodes=0
%%%mzn-stat: openNodes=-1
%%%mzn-stat: solveTime=0.0012
%%%mzn-stat-end
```

Expected Output

```
assignment: [2, 1, 3, 4]
total cost: 13
----------
==========
%%%mzn-stat: objective=13
%%%mzn-stat: objectiveBound=13
%%%mzn-stat: nodes=1
%%%mzn-stat: solveTime=0.0044
%%%mzn-stat-end
```

Step 8 (continued)



Caution

As a final counter-example, try the same command above with `--solver Gurobi` in place of `--solver HiGHS` on this unedited file pair. It fails with the identical `No Gurobi license found` error you already saw during the Setup section's probe — Gurobi is *registered* for this model exactly as Gecode, COIN-BC and HiGHS are, but registration alone never guarantees a working solve. This is not a required working step; it is evidence, not a fourth comparison point.

Checkpoint — verify before moving on

Gecode, COIN-BC and HiGHS all report total cost 13 for the identical, unedited `assignment.mzn/assignment.dzn` pair, with three different statistics vocabularies and (possibly) different reported pairings. The figure above plots exactly this data. You have now confirmed solver independence with your own hands, on your own machine.

5 Unit 4 — MiniZinc from Python; Choosing a Solver

(*See Course Notes Week 6, Unit 4 — MiniZinc from Python; Choosing a Solver.*)

Step 9: Read minizinc_python.py's Model/Instance/Solver.lookup pattern and run it

What will you do? Open `modeller/minizinc_python.py` and locate the three pieces shown below: loading a `Model`, building a `Gecode Instance` and solving it, and looping over candidate MIP backend names with a `try/except` around each solve. Then, with the course's shared virtual environment active, run the script.

Why? This is Unit 4's whole point demonstrated in code: the exact comparison you just ran three times by hand from the command line — Gecode, then COIN-BC, then HiGHS, each a separate typed command — is now one Python script that also, safely, tries Gurobi and reports it as unavailable instead of crashing, exactly the "listed versus usable" distinction

Step 9 (continued)

Unit 3 introduced.

Loading a model — one `Model` object per problem:

```
knapsack_model = minizinc.Model(MZN_DIR / "knapsack.mzn")
knapsack_model.add_file(MZN_DIR / "knapsack.dzn")
nqueens_model = minizinc.Model(MZN_DIR / "nqueens.mzn")
nqueens_model.add_file(MZN_DIR / "nqueens.dzn")
```

Building a Gecode `Instance` and solving it, timed:

```
solver = minizinc.Solver.lookup("gecode")
instance = minizinc.Instance(solver, model)
start = time.perf_counter()
result = instance.solve()
elapsed_ms = (time.perf_counter() - start) * 1000
rows.append(("Gecode", str(result.status), elapsed_ms, summarize(result)))
```

Looping over candidate MIP backends, skipping unavailable ones safely:

```
for backend in MIP_BACKENDS:
   try:
      solver = minizinc.Solver.lookup(backend)
      instance = minizinc.Instance(solver, model)
      start = time.perf_counter()
      result = instance.solve()
      elapsed_ms = (time.perf_counter() - start) * 1000
      rows.append((backend, str(result.status), elapsed_ms, summarize(result)))
   except minizinc.MiniZincError as error:
      rows.append((backend, "unavailable", 0.0, str(error).splitlines()[0]))
```

Terminal

```
$ source ../.venv/bin/activate
$ python modeller/minizinc_python.py
```

Expected result: Two comparison tables, one for knapsack and one for N-queens, each with four rows (Gecode, coin-bc, highs, gurobi). Gecode, coin-bc and highs report a solved status; gurobi reports `unavailable` with its captured licence-error reason. Your own timing column will differ from the figures below — wall-clock time is never guaranteed to repeat — but the status column and the reported objective/solution must match exactly.

Expected Output

```
knapsack (n=4, capacity=7):
solver     status                 time_ms  summary
Gecode     OPTIMAL_SOLUTION        117.83  objective=9
coin-bc    OPTIMAL_SOLUTION        220.22  objective=9
highs      OPTIMAL_SOLUTION        221.70  objective=9
gurobi     unavailable               0.00    MIPGurobiWrapper runtime
    error:  No Gurobi license found

nqueens (n=6):
solver     status                 time_ms  summary
Gecode     SATISFIED               125.09  solution found
coin-bc    SATISFIED               238.88  solution found
highs      SATISFIED               237.11  solution found
gurobi     unavailable               0.00    MIPGurobiWrapper runtime
    error:  No Gurobi license found
```

Step 9 (continued)

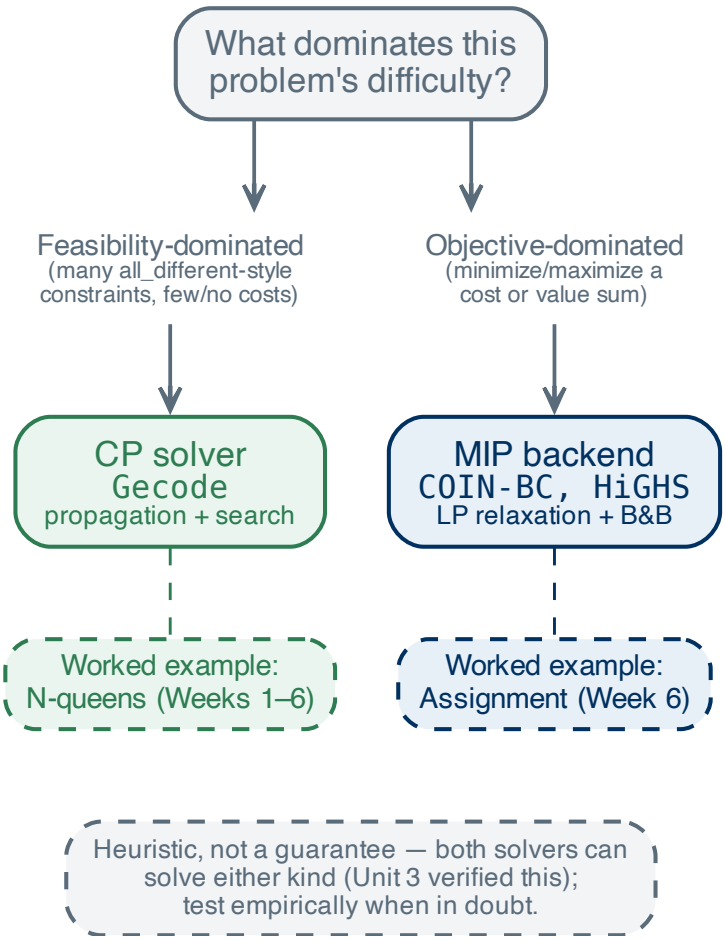
Tip

The `summary` column reads differently for the two problems on purpose: knapsack has an objective (`objective=9`), while N-queens is a pure satisfaction problem with no objective at all — `summarize()` in the script checks for an `objective` attribute and falls back to `"solution found"` when there is none, the same distinction Unit 1's `solve satisfy;` versus `solve maximize ...;` makes at the model level.

Step 10: Apply the CP-vs-MIP decision guide to N-queens and assignment

What will you do? Study the figure below. In three sentences or fewer, state whether N-queens (a pure satisfaction problem, all `all_different` constraints, no objective) or the assignment problem (one linear cost sum to minimize, one feasibility constraint) is the more natural fit for a CP solver like Gecode, and which is the more natural fit for a MIP backend — justify your answer using the guide's two structural signals.

Why? This is Unit 4's own practice task, applied here on paper before you use the same reasoning, unaided, on a brand-new problem in this sheet's capstone step.



Expected result: N-queens is the more natural CP fit — a pure satisfaction problem dominated by combinatorial `all_different`-style constraints with no linear objective at all,

Step 10 (continued)

exactly Gecode's propagation-plus-search strength. The assignment problem is the more natural MIP fit — its feasibility constraint is simple, but its defining feature is a linear objective over a cost matrix, exactly what LP relaxation and branch-and-bound are built for. Both solvers can, and in Unit 3 did, solve either kind correctly; this is a question about which structure each solver is naturally suited to, not about which one works.

☑ Checkpoint — verify before moving on

`minizinc_python.py`'s two comparison tables both report Gecode, coin-bc and highs solving correctly and gurobi as unavailable, and you have applied the decision guide's two structural signals — feasibility-dominated versus objective-dominated — to N-queens and assignment on paper. You are ready to apply the same reasoning to a problem this book has not modeled for you.

6 Capstone — Build a Complete Model From Scratch

Step 11: Build a lab-kit coverage model from scratch and verify it on Gecode

What will you do? Your department shares five measurement kits across four lab stations. Each kit covers only some stations (it must be physically compatible with that station's equipment) and has its own rental cost; the table below lists which stations each kit covers and what it costs. Choose the cheapest subset of kits that covers every station at least once — a station may end up covered by more than one selected kit, and that is fine. Build a complete MiniZinc model for this problem entirely on your own, in a new file pair `lab/kit_cover.mzn` and `lab/kit_cover.dzn` (create the `lab` folder inside `kod_ornekleri/hafta06` if it does not already exist) — no starter file this time. Then solve it on Gecode.

Kit	Covers	Cost
K1	S1, S2	5
K2	S2, S3	3
K3	S3, S4	4
K4	S1, S4	7
K5	S1, S3	4

Why? This is this week's integrative step (H5): naming which of this new problem's quantities are parameters and which are decision variables — without a paper model or a starter file handed to you first — is exactly the transfer Unit 1's closing note promises, applied here to a lab-resource-sharing instance shaped like nothing else you modeled this week.

Expected result: Choosing kits K1 and K3 covers all four stations (K1 gives S1 and S2, K3 gives S3 and S4) at total cost 9 — the model's unique optimum, since every other feasible combination of kits costs 10 or more.

```
>_ Terminal
$ minizinc --solver Gecode lab/kit_cover.mzn lab/kit_cover.dzn
```

Step 11 (continued)

Expected Output

```
chosen kits: [true, false, true, false, false]
total cost: 9
```

Tip

Follow the same four-block shape every model this week has used: parameters first (station and kit counts, a Boolean coverage table, and the cost list), then one Boolean decision variable per kit, then one constraint per station requiring at least one selected kit to cover it (a `forall` over stations wrapping a `sum` over kits), and a single `solve minimize` item over the chosen kits' costs — Unit 1's four building blocks, applied to a problem you have not seen modeled before.

Common Pitfall

Writing the coverage requirement as an equality (exactly one selected kit per station) instead of "at least one" over-constrains the model: a station legitimately covered by two selected kits at once would then make the whole instance infeasible, even though double-covering a station is perfectly acceptable in a set-cover problem. Only *under*-coverage should ever be forbidden.

Step 12: Run it unchanged on a MIP backend and justify your solver choice

What will you do? Without editing `kit_cover.mzn` or `kit_cover.dzn`, run the same file pair once more with `--solver COIN-BC` (or `--solver HiGHS`). Then, in one sentence, justify — in the decision guide's own structural terms — which solver you would choose for this problem if you only got to run it once.

Why? This closes the loop from Unit 3's solver-independence guarantee (same optimal value, no edits) to Unit 4's structural decision guide, applied together to a model you built entirely yourself — exactly what this week's integrative objective asks for.

Terminal

```
$ minizinc --solver COIN-BC lab/kit_cover.mzn lab/kit_cover.dzn
```

Expected Output

```
chosen kits: [true, false, true, false, false]
total cost: 9
```

Expected result: The same total cost, 9, confirming solver independence on a model you wrote yourself. For the justification, a reasonable answer notes that this problem carries *both* of the decision guide's signals at once — a combinatorial, constraint-heavy coverage requirement (favouring Gecode) and a linear cost objective to minimize (favouring a MIP backend) — so either solver is a defensible choice at this size; what should not appear in your answer is a claim that only one of the two solvers can solve this model correctly, since Unit 3 already showed both agreeing on the optimum.

☑ Checkpoint — verify before moving on

Your own `kit_cover.mzn` reports total cost 9 — kits K1 and K3 — on both Gecode and your chosen MIP backend, with no edits between the two runs, and you have written one sentence justifying a solver choice in the decision guide's own structural terms. This is the same parameters-variables-constraints-solve pattern you have now built or run five times today, the last time entirely from a written description rather than a file someone else prepared for you.

Troubleshooting

Symptom	Likely cause	Fix
`minizinc: command not found`	The Homebrew install did not finish, or your shell has not picked up the new `PATH` entry	Re-run `brew install minizinc`, then open a new terminal window before trying again
`minizinc --solver HiGHS` reports a plugin or library load error	`brew install highs` was skipped, or the `Preferences.json` entry is missing or misspelled	Re-check both parts of the Setup section's HiGHS step; the file path must match exactly what `brew install highs` placed on your own machine
`Error: undefined identifier all_different` during `--compile` or a solve	Your own `.mzn` file is missing `include "all_different.mzn";` at the top — MiniZinc 2.10 no longer auto-includes it	Add the `include` line as the first non-comment line of the file, exactly as `nqueens.mzn` and `assignment.mzn` both do
`ModuleNotFoundError: No module named 'minizinc'` when running `minizinc_python.py`	The Week 1 shared virtual environment is not active in this terminal	Re-run `source ../.venv/bin/activate` (or `Activate.ps1` on Windows) from inside `kod_ornekleri/hafta06`
Your own `kit_cover.mzn` reports `UNSATISFIABLE`	The coverage constraint was written as an equality instead of "at least one," or the `covers` table was transcribed with kit and station indices swapped	Re-read the `ortakhata` box in this section; confirm `covers[k,s]` indexes kit `k` first and station `s` second, matching the table exactly
`minizinc --solver Gurobi ...` fails with `No Gurobi license found`	Expected in this course's shared environment — the academic licence MiniZinc's own Gurobi plugin needs is not installed here	Do not debug your model; this is a documented environment limitation from the Setup section's solver-inventory step, not a modeling mistake

5 • Follow-through **↱ Transfer — Where Else?**

The three files you ran unchanged across solvers today — `knapsack.mzn`, `nqueens.mzn` and `assignment.mzn` — are not disposable lab work: Week 7 opens `gurobipy` directly and reuses these exact same knapsack and assignment cost figures as its own starting instances, now built again from the mathematical-programming side instead of through MiniZinc. More generally, the habit you practiced today — write a model once, in a language whose only job is describing it, and only afterward decide which program actually searches for a solution — is this course's first, smallest example of separating "is the model correct" from "which engine should solve it," a distinction you will keep leaning on every time this course moves between CP-SAT-style

and MIP-style views of the same problem, and in any future modeling work outside this course where you cannot be certain, ahead of time, which solver a colleague or a deployment environment will actually have installed.